

Стать IT-шником может каждый



Python DA Summary 2



Преподаватель



Ваше фото

ИМЯ ФАМИЛИЯ

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)



ВАЖНО!

- Камера должна быть включена на протяжении всего занятия.
- Если у Вас возник вопрос в процессе занятия, пожалуйста, поднимите руку и дождитесь, пока преподаватель закончит мысль и спросит Вас, также можно задать вопрос в чате или когда преподаватель скажет, что начался блок вопросов.
- Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях.
- Вести себя уважительно и этично по отношению к остальным участникам занятия.
- Во время занятия будут интерактивные задания, будьте готовы включить микрофон или демонстрацию экрана по просьбе преподавателя.

ПЛАН ЗАНЯТИЯ

Повторение изученного за неделю
Вопросы по повторению
Разбор домашних заданий
Вопросы по разбору заданий
Задание для закрепления
Оставшиеся вопросы



Стать IT-шником может каждый



ПОВТОРЕНИЕ ИЗУЧЕННОГО



Стать IT-шником может каждый



Экспресс-опрос



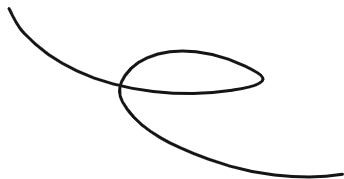
Что такое векторы?





Векторы —

это одномерные массивы. Возникают в практических задачах, когда мы имеем дело с упорядоченным набором чисел.



Векторы в практических задачах

```
# Вектор ежедневных расходов (в евро)
daily_expenses = np.array([10, 20, 15, 10, 30, 50, 50])

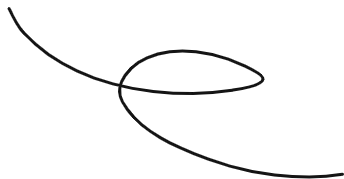
# Температура воздуха по месяцам (в градусах Цельсия)
temperature = np.array([1, 2, 6, 11, 16, 19, 21, 21, 16, 11, 6, 2])
```

Полезно знать



Операции над векторами —

это операции преобразования компонентов этих векторов.



Операции над векторами

```
# Перевод ежедневных расходов в доллары  
print(daily_expenses * 1.1)  
# [11. 22. 16.5 11. 33. 55. 55.]
```

```
# Перевод температуры в Фаренгейты  
print(1.8 * temperature + 32)  
# [33.8 35.6 42.8 51.8 60.8 66.2 69.8 69.8 60.8 51.8 42.8 35.6]
```

Стать IT-шником может каждый



Экспресс-опрос



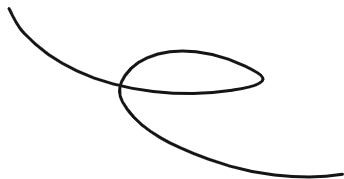
Что такое матрицы?





Матрицы —

это двумерные массивы. Возникают в практических задачах, когда нам нужно работать с несколькими родственными упорядоченными наборами чисел.



Матрицы в практических задачах

```
# Доходы членов семьи по месяцам
income_matrix = np.array([[2000, 3000, 3500, 3000, 3000, 2500, 3500, 2000, 2500, 3000,
3500, 4000], [1500, 2500, 6000, 1100, 1600, 1900, 2100, 4200, 6000, 1100, 6000, 2000]])

print(income_matrix)

# [[2000 3000 3500 3000 3000 2500 3500 2000 2500 3000 3500 4000]
#  [1500 2500 6000 1100 1600 1900 2100 4200 6000 1100 6000 2000]]
```

Пример

Кондитерская производит несколько видов десертов.

Для каждого необходимы мука, яйца и сахар, но в разном количестве.

Необходимое количество описать матрицей, в которой каждая строка описывает одну партию какого-то десерта, при этом первый столбец показывает количество муки в килограммах, второй столбец – количество яиц в штуках, третий – количество сахара в килограммах.

Пример

```
# Расход продуктов
goods = np.array([[0.5, 6, 0.1],
                  [1, 10, 0.5],
                  [0.3, 10, 0.6],
                  [0.5, 7, 0.3]])

print(goods)
# [[ 0.5  6.  0.1]
#  [ 1. 10.  0.5]
#  [ 0.3 10.  0.6]
#  [ 0.5  7.  0.3]]
```

Нахождение суммарных затрат

$$\begin{bmatrix} 0.5 & 6 & 0.1 \\ 1 & 10 & 0.5 \\ 0.3 & 10 & 0.6 \\ 0.5 & 7 & 0.3 \end{bmatrix} * \begin{bmatrix} 0.75 \\ 0.22 \\ 0.88 \end{bmatrix} =$$

$$= \begin{bmatrix} (0.5 \times 0.75) + (6 \times 0.22) + (0.1 \times 0.88) \\ (1 \times 0.75) + (10 \times 0.22) + (0.5 \times 0.88) \\ (0.3 \times 0.75) + (10 \times 0.22) + (0.6 \times 0.88) \\ (0.5 \times 0.75) + (7 \times 0.22) + (0.3 \times 0.88) \end{bmatrix} = \begin{bmatrix} 1.783 \\ 3.39 \\ 2.953 \\ 2.179 \end{bmatrix}$$

Нахождение суммарных затрат (Python)

```
prices_col = prices.reshape(3,1)
expenses = goods @ prices_col
print(expenses)
print(expenses.sum())
```

```
# [[1.783]
# [3.39 ]
# [2.953]
# [2.179]]
# 10.305000000000001
```

Стать IT-шником может каждый



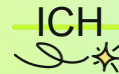
Экспресс-опрос



Что такое система уравнений?

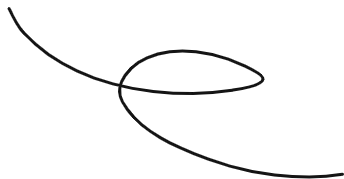


Полезно знать



Система уравнений —

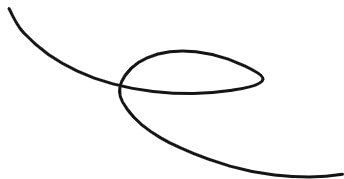
это набор уравнений с несколькими переменными.





Решение системы уравнений —

это набор значений переменных, при подстановке которых в уравнения все они обращаются в верные равенства.



Пример

Фирма закупает ноутбуки и стационарные компьютеры для оборудования офисов на двух этажах здания.

- На первом этаже планируется поставить 20 стационарных компьютеров и 20 ноутбуков.
- На втором этаже – 12 стационарных компьютеров и 35 ноутбуков.

Стоимость оборудования первого этажа должна составить 40000 евро, а второго этажа – 50000.

Обозначим через X - стоимость одного ПК, y - стоимость одного ноутбука.

Пример

$$\begin{cases} 20x + 20y = 40000 \\ 12x + 35y = 50000 \end{cases}$$

$$A = \begin{bmatrix} 20 & 20 \\ 12 & 35 \end{bmatrix}, \quad X = \begin{bmatrix} x \\ y \end{bmatrix}, \quad B = \begin{bmatrix} 40000 \\ 50000 \end{bmatrix}$$

$$AX = B$$

$$X = \frac{1}{A} B = A^{-1} B$$

Стать IT-шником может каждый



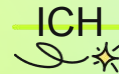
Экспресс-опрос



Что такое обратные числа?

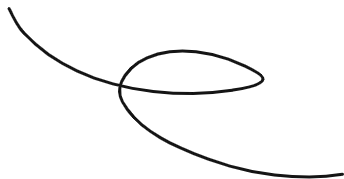


Полезно знать



Обратные числа —

это числа, произведение которых равно единице.



Экспресс-опрос



Что такое взаимно обратные матрицы?

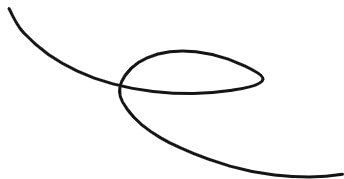


Полезно знать



Взаимно обратные матрицы —

это матрицы, произведение которых равно единичной матрице.

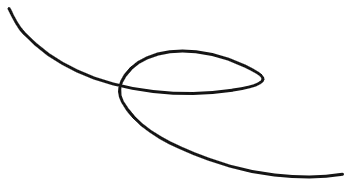


Полезно знать



`linalg.inv(matrix)` —

это метод в `numpy` для нахождения обратной матрицы к квадратной.



Метод `linalg.inv(matrix)`

```
equipment = np.array([[20, 20], [12, 35]])  
  
inverse = np.linalg.inv(equipment) # обратная матрица  
  
print(inverse)  
print(inverse @ equipment)  
print(equipment @ inverse)
```

Метод `linalg.inv(matrix)`

```
equipment = np.array([[20, 20], [12, 35]])  
expenses = np.array([[40000], [50000]])  
inverse = np.linalg.inv(equipment)  
print(inverse @ expenses)  
# [[ 869.56521739]  
# [1130.43478261]]
```

Пример

Фирма закупает ноутбуки и стационарные компьютеры для оборудования офисов на двух этажах здания.

- На первом этаже планируется поставить 20 стационарных компьютеров и 20 ноутбуков.
- На втором этаже – 30 стационарных компьютеров и 30 ноутбуков.

Стоимость оборудования первого этажа должна составить 40000 евро, а второго этажа – 50000.

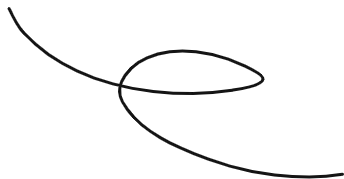
Нахождение обратной матрицы

```
equipment = np.array([[20, 20], [30, 30]])  
inverse = np.linalg.inv(equipment)  
print(inverse)  
  
# numpy.linalg.LinAlgError: Singular matrix
```



Singular matrix —

это ошибка в `numpy`, которая появляется, если найти обратную матрицу невозможно - если исходная матрица была квадратной.



Экспресс-опрос



Что такое взаимно определитель?

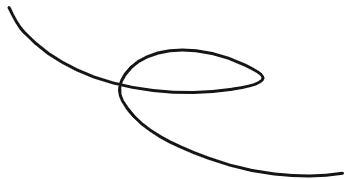




Определитель —

это специальная числовая характеристика матрицы.

Система уравнений с данной матрицей имеет единственное решение, только если ее определитель не равен нулю.



Стать IT-шником может каждый



Экспресс-опрос



Что такое `linalg.det(matrix)`?

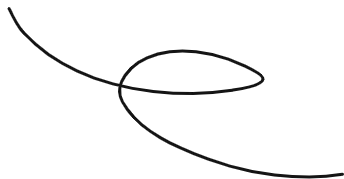


Полезно знать



`linalg.det(matrix)` —

это метод в `numpy` для нахождения определителя.



Нахождение определителя

```
equipment = np.array([[20, 20], [30, 30]])  
det = np.linalg.det(equipment)  
  
print(det)
```

Стать IT-шником может каждый



ВОПРОСЫ



Скорость NumPy



- NumPy эффективно использует память для хранения массивов.
- Операции выполняются на уровне C, что минимизирует накладные расходы интерпретатора Python.
- Элементы массива хранятся в памяти последовательно и имеют фиксированный размер, NumPy может точно вычислить, где в памяти находится каждый элемент.

Сравнение скорости работы

```
N = 5000
list_2d = [[i + N*j for i in range(N)] for j in range(N)]
array_2d = np.array(list_2d)

%%timeit
list_2d_mult = [[list_2d[i][j] * list_2d[i][j] for j in range(N)] for i in range(N)]

%%timeit
arr = []
for i in range(N):
    subarr = []
    for j in range(N):
        subarr.append(list_2d[i][j] * list_2d[i][j])
    arr.append(subarr)

%%timeit
array_2d_mult = array_2d * array_2d
```

Сравнение скорости работы

```
N = 5000
list_2d = [[i + N*j for i in range(N)] for j in range(N)]
array_2d = np.array(list_2d)
```

```
%%timeit
list_2d_mult = [[list_2d[i][j] * list_2d[i][j] for j in range(N)] for i in range(N)]
```

```
%%timeit
arr = []
for i in range(N):
    subarr = []
    for j in range(N):
        subarr.append(list_2d[i][j] * list_2d[i][j])
    arr.append(subarr)
```

```
%%timeit
array_2d_mult = array_2d * array_2d
```

Стать IT-шником может каждый



Экспресс-опрос



Что такое "C-style"?

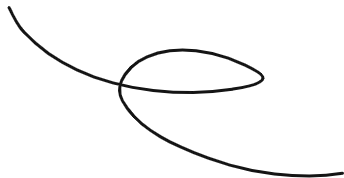


Полезно знать



"C-style" —

это порядок хранения данных, который означает последовательное расположение по последнему измерению (по строкам).



Стать IT-шником может каждый



Экспресс-опрос



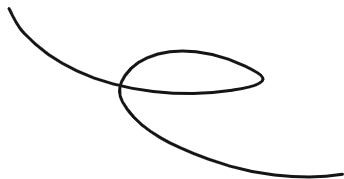
Что такое "Fortran-style"?





"Fortran-style" —

это порядок хранения данных, который означает последовательное расположение по первому измерению (по столбцам).



Базовые функции библиотеки numpy.random

Функция	Пояснение
<code>randint(low[, high, size, dtype])</code>	Случайное число или массив чисел в диапазоне
<code>rand(size)</code>	Создает массив указанного размера со значениями от 0 до 1
<code>choice(a[, size, replace, p])</code>	Генерирует случайную выборку из заданного одномерного массива
<code>shuffle(x)</code>	Изменяет массив, перемешивая его значения
<code>permutation(x)</code>	Перемешивает значения массива и возвращает его перемешанную копию, исходный массив не изменяется

Создание массива случайных чисел из 5 элементов со значениями из интервала от 0 до 1

```
print(np.random.rand(5))  
# [0.09396291 0.80335857 0.42652831 0.63249204 0.15404466]
```

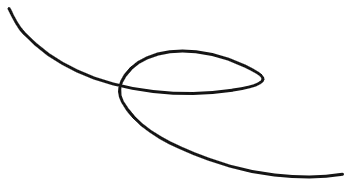
Создание массива случайных целых чисел размером 5×5 в диапазоне от 10 до 20 (оба включительно)

```
print(np.random.randint(10, 20, (5, 5)))  
# [[19 19 16 18 13]  
#  [16 10 13 10 14]  
#  [14 15 11 11 17]  
#  [17 19 19 18 18]  
#  [11 12 10 15 18]]
```



Распределение —

это математическое представление, которое показывает все возможные значения (или интервалы) случайной переменной и частоту (вероятность) их появления.



Устройство многомерных массивов в памяти

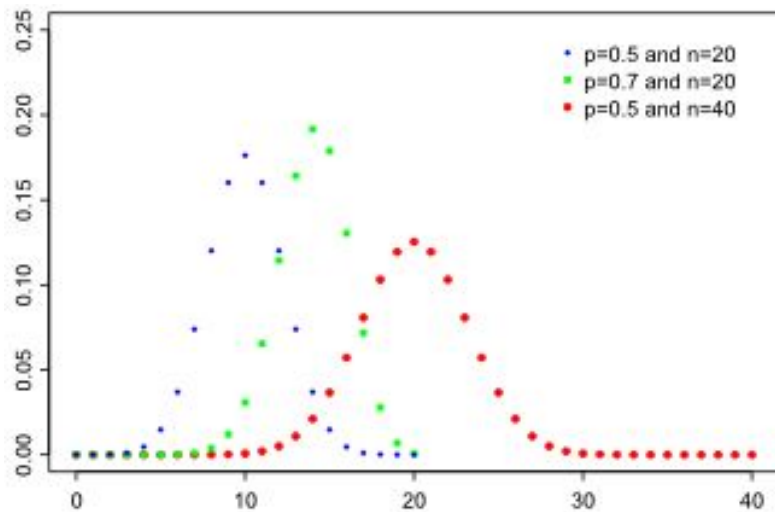
Дискретные распределения

Описывают случаи, где переменная может принимать только определенные, отдельные значения

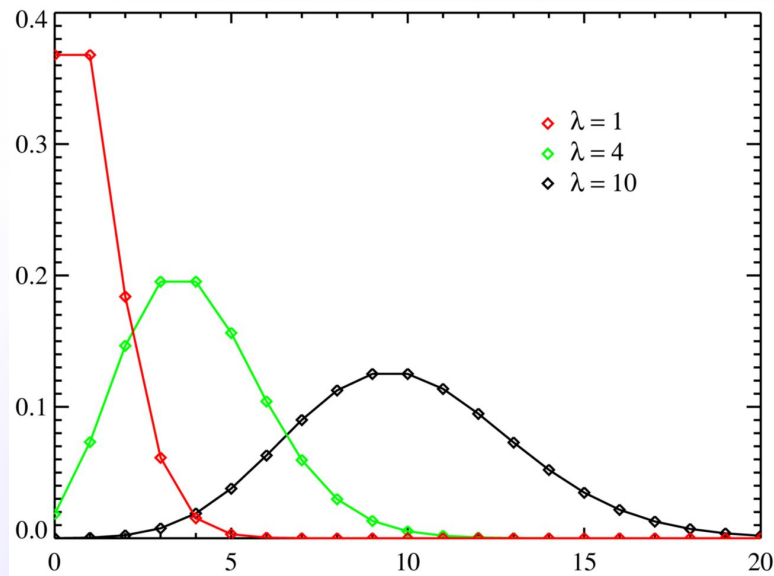
Непрерывные распределения

Описывают случаи, где переменная может принимать любое значение в заданном интервале

Дискретные распределения

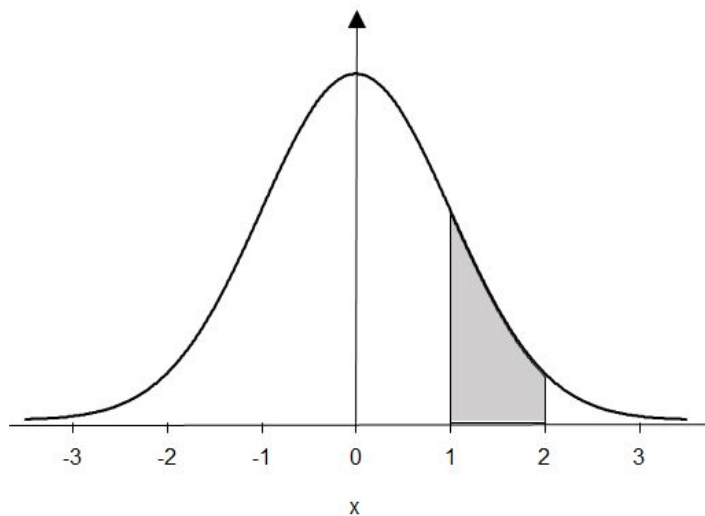


Биномиальное

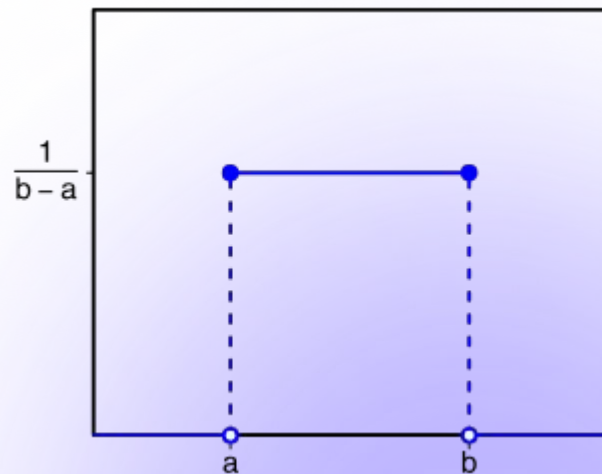


Пуассоновское

Непрерывные распределения



Нормальное



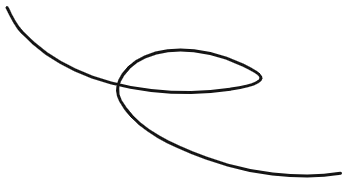
Равномерное



Биномиальное распределение —

это **распределение**, которое описывает количество успехов в последовательности из n независимых экспериментов, где вероятность успеха в каждом эксперименте одинакова и равна p .

```
binomial(n, p[, size])
```

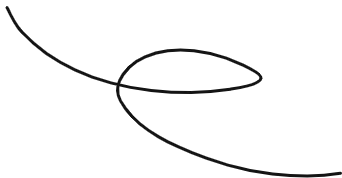




Нормальное распределение —

это **распределение**, самое известное распределение в статистике, описывается колоколообразной кривой. Определяется двумя параметрами: средним (loc) и стандартным отклонением (scale).

```
normal([loc, scale, size])
```

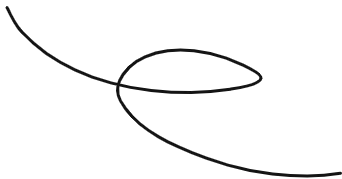




Нормальное распределение —

это распределение, которое описывает количество событий, происходящих за фиксированный интервал времени, если эти события происходят с постоянной средней скоростью и независимо друг от друга.

```
poisson([lam, size])
```

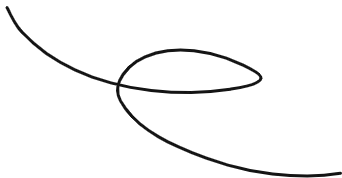




Равномерное распределение —

это **распределение**, в котором все события имеют одинаковую вероятность. Определяется нижней (low) и верхней (high) границами.

```
uniform([low, high, size])
```



Генерация случайных чисел из конкретных распределений

```
print(np.random.normal(size=5))  
# [ 0.01660124  0.25662967  0.11505522  0.70052572 -0.04406528]
```


Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



Экспресс-опрос



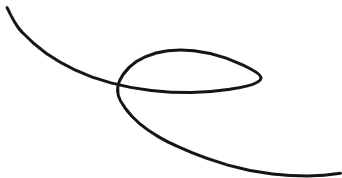
Что такое Pandas?





Pandas —

это мощная библиотека для анализа и манипуляции данными, написанная на языке программирования Python. Она предоставляет удобные и быстрые инструменты для работы с табличными данными: таблицы баз данных SQL, Excel-файлы и CSV-файлы.





Установка библиотеки pandas

```
pip install pandas
```



Подключение библиотеки pandas

```
import pandas as pd
```

Стать IT-шником может каждый



Экспресс-опрос



Что такое Series?

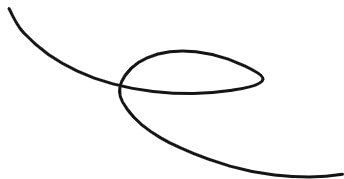


Полезно знать



Series —

это одномерный массив данных.



Экспресс-опрос



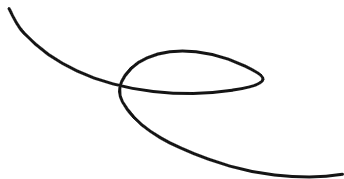
Что такое DataFrame?





DataFrame —

это двумерная структура данных, похожая на таблицу в базе данных или на лист в Excel, которая состоит из строк и столбцов.



Библиотека pandas

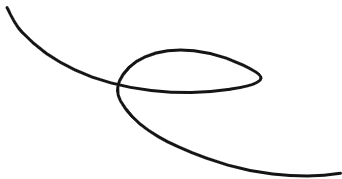


- Широко используется в научных исследованиях, финансовом анализе, инженерии, статистике, машинном обучении и многих других областях, где необходима работа с данными.
- Позволяет аналитикам и разработчикам эффективно обрабатывать большие объемы данных, проводить комплексный анализ, строить отчеты и препроцессинг данных для машинного обучения.



Series —

это одномерный массив данных, который может хранить данные любого типа. Элементы в Series индексируются, по умолчанию используется числовой индекс от 0 до N-1, где N – количество элементов в Series. Pandas позволяет задать и использовать именованные индексы.





Особенности Series

- Series поддерживает автоматическое индексирование и возможность задавать пользовательские индексы.
- Пользовательские индексы могут быть числами, строками или даже датами.
- Каждому элементу в Series можно присвоить метку (через индекс).
- Множество встроенных методов для анализа данных: статистические функции, операции сортировки и ранжирования, методы для обработки пропущенных значений.
- Series - строительный блок для DataFrame в pandas.
- DataFrame - контейнер для Series.

Пример создания Series

```
import numpy as np
import pandas as pd

l = [1, 3, 5, np.nan, 6, 8]
s = pd.Series(l)
s
0    1.0
1    3.0
2    5.0
3    NaN
4    6.0
5    8.0
dtype: float64
```

Пример создания Series

```
s.values, type(s.values)
# (array([ 1.,  3.,  5., nan,  6.,  8.]), numpy.ndarray)
s.to_dict()
# {0: 1.0, 1: 3.0, 2: 5.0, 3: nan, 4: 6.0, 5: 8.0}
s.index
# RangeIndex(start=0, stop=6, step=1)
```

Пример создания Series

```
s.describe()
```

```
count    5.000000  
mean     4.600000  
std      2.701851  
min      1.000000  
25%      3.000000  
50%      5.000000  
75%      6.000000  
max      8.000000  
dtype: float64
```

Индексация

```
s = pd.Series([1, 3, 5, np.nan, 6, 8])
```

```
s[2]
```

```
s[2] = 7
```

```
s
```

```
s[2:5]
```

```
s1 = s[1:]
```

```
s1
```

```
s2 = s[:-1]
```

```
s2
```

Индексация

```
s1 + s2
```

```
0 NaN
```

```
1 6.0
```

```
2 14.0
```

```
3 NaN
```

```
4 12.0
```

```
5 NaN
```

```
dtype: float64
```


Индексация

```
np.exp(s1)  
type(np.exp(s)), type(s)
```

Индексация по умолчанию

S.index

RangeIndex(start=0, stop=6, step=1)

type(np.exp(s)), type(s)

Наборы данных с индексом, заданным СПИСКОМ

```
i = list('abcdef') # i == ['a', 'b', 'c', 'd', 'e', 'f']
s = pd.Series(l, index=i)
s
```

a	1.0
b	3.0
c	5.0
d	NaN
e	6.0
f	8.0

dtype: float64

Индексация

S['c']

5.0

s[5]

8.0

s.c

5.0

Создание набора данных из словаря

```
s = pd.Series({'a':1, 'b':2, 'c':0})  
s
```

```
a    1  
b    2  
c    0  
dtype: int64
```

Последовательность дат как индекс

```
d = pd.date_range('20160101', periods=6)
d
# DatetimeIndex(['2016-01-01', '2016-01-02', '2016-01-03', '2016-01-04',
#               '2016-01-05', '2016-01-06'],
#               dtype='datetime64[ns]', freq='D')
```

```
s = pd.Series(l, index=d)
```

```
s
```

```
2016-01-01    1.0
2016-01-02    3.0
2016-01-03    5.0
2016-01-04    NaN
2016-01-05    6.0
2016-01-06    8.0
Freq: D, dtype: float64
```

```
s[s > 3]
```

```
2016-01-03    5.0
2016-01-05    6.0
2016-01-06    8.0
dtype: float64
```

Стать IT-шником может каждый



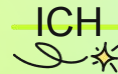
Экспресс-опрос



Что такое diff?

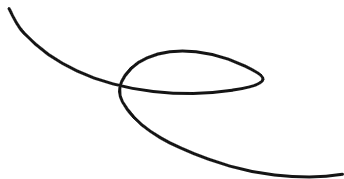


Полезно знать



diff —

это метод, который позволяет посчитать разности соседних элементов.



Стать IT-шником может каждый



Экспресс-опрос



Что такое periods?



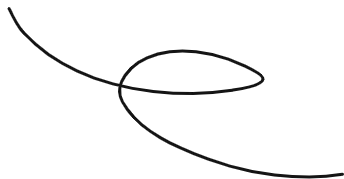
Полезно знать



ICH
e *

periods —

это **ключевое слово**, который отвечает за то, с каким шагом будут считаться разности.



Метод diff

```
s.diff()
```

```
2016-01-01    NaN
2016-01-02    2.0
2016-01-03    2.0
2016-01-04    NaN
2016-01-05    NaN
2016-01-06    2.0
Freq: D, dtype: float64
```

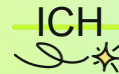
Экспресс-опрос



Что такое dropna?

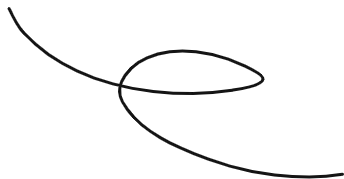


Полезно знать



dropna —

это метод, который позволяет избавиться от пропущенных значений.



Метод dropna

```
s.diff().dropna()
```

```
2016-01-02    2.0  
2016-01-03    2.0  
2016-01-06    2.0  
dtype: float64
```

Кумулятивные минимумы

```
s.cummin()
```

```
2016-01-01    1.0  
2016-01-02    1.0  
2016-01-03    1.0  
2016-01-04    NaN  
2016-01-05    1.0  
2016-01-06    1.0  
Freq: D, dtype: float64
```

Кумулятивные суммы

```
s.cumsum()
```


Стать IT-шником может каждый



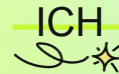
Экспресс-опрос



Что такое expanding?

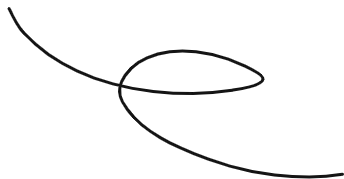


Полезно знать



expanding —

это конструкция, которая позволяет считать произвольные функции кумулятивным способом.



Конструкция expanding

```
s.expanding().apply(np.median)
```

Стать IT-шником может каждый



ВОПРОСЫ



Экспресс-опрос



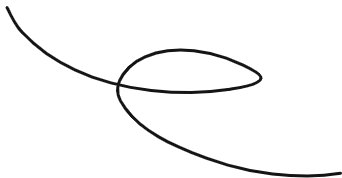
Что такое DataFrame?





DataFrame —

это двумерная таблица с данными. Имеет индекс и набор столбцов (возможно, имеющих разные типы). При этом и каждый столбец, и каждая строка будут Series.



Создание DataFrame

```
d = {'one': pd.Series(range(6), index=list('aaefg'))}  
df = pd.DataFrame(d)  
df
```

```
#      one  
# a      0  
# a      1  
# a      2  
# e      3  
# f      4  
# g      5
```

Экспресс-опрос



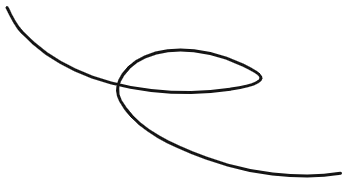
Что такое display: display(df)?





display: display(df) —

это инструкция, которую используют для визуализации датафрейма в jupyter notebook / colab.



Сброс индекса

```
df.reset_index()
#   index one
#0     a   0
#1     a   1
#2     a   2
#3     e   3
#4     f   4
#5     g   5
```

Датафрейм с несколькими столбцами

```
d = {'one': pd.Series(range(6), index=list('abdefg')),  
      'two': pd.Series(range(7), index=list('abcdefg')),  
      'three': pd.Series(np.random.normal(size=7), index=list('abcdefg'))}
```

```
df = pd.DataFrame(d)
```

```
df
```

#	one	two	three
#a	0.0	0	-1.260839
#b	1.0	1	-0.380510
#c	NaN	2	1.370570
#d	2.0	3	0.302637
#e	3.0	4	1.690632
#f	4.0	5	-0.757313
#g	5.0	6	0.560045

Датафрейм с несколькими столбцами

```
df['one']  
#a  0.0  
#b  1.0  
#c  NaN  
#d  2.0  
#e  3.0  
#f  4.0  
#g  5.0  
#Name: one, dtype: float64
```

Датафрейм с несколькими столбцами

```
df.index  
#Index(['a', 'b', 'c', 'd', 'e', 'f', 'g'], dtype='object')
```

Таблица с несколькими разными типами данных

```
df2 = pd.DataFrame({ 'A': 1.,  
                     'B': pd.Timestamp('20130102'),  
                     'C': pd.Series(1, index=list(range(4)),  
                                   dtype='float32'),  
                     'D': np.array([3] * 4,  
                                   dtype='int32'),  
                     'E': pd.Categorical(["test", "train",  
                                           "test", "train"]), # one - hot encoding  
                     'F': 'foo' })
```

df2	A	B	C	D	E	F
#	A	B	C	D	E	F
#0	1.0	2013-01-02	1.0	3	test	foo
#1	1.0	2013-01-02	1.0	3	train	foo
#2	1.0	2013-01-02	1.0	3	test	foo
#3	1.0	2013-01-02	1.0	3	train	foo

Таблица с несколькими разными типами данных

```
df2.dtypes  
#A      float64  
#B  datetime64[ns]  
#C      float32  
#D       int32  
#E     category  
#F       object  
#dtype: object
```

Анализ DataFrame

```
df.head() # в аргументе можем указать сколько первых строк хотим вывести  
df.tail(3) # (а здесь последних). по умолчанию – 5  
df.index # индексы  
df.columns # названия колонок
```


Анализ DataFrame

```
df.values # получение обычной матрицы данных
#array([[ 0.        ,  0.        , -1.26083858],
#       [ 1.        ,  1.        , -0.38050995],
#       [ nan,  2.        ,  1.37057007],
#       [ 2.        ,  3.        ,  0.30263747],
#       [ 3.        ,  4.        ,  1.69063229],
#       [ 4.        ,  5.        , -0.75731332],
#       [ 5.        ,  6.        ,  0.56004507]])
```

Описательные статистики

```
df.describe()
```

Транспонирование данных

```
df.T
#      a      b      c      d      e      f      g
#one  0.0000  1.0000 NaN    2.0000  3.0000  4.0000  5.0000
#two  0.0000  1.0000  2.0000  3.0000  4.0000  5.0000  6.0000
#three -1.261 -0.380  1.3705  0.3026  1.6906 -0.757  0.5600
```

Сортировка по столбцу

```
df.sort_values(by='three', ascending=False)
```

Итерирование по столбцам

```
for i in df.columns:  
    print(i, df[i])
```

Итерирование по строкам

```
for i, j in df.iterrows():  
    print(i, j)
```

Стать IT-шником может каждый



Экспресс-опрос



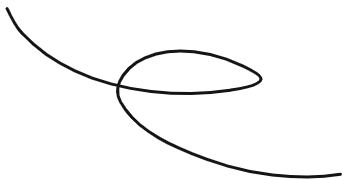
Что такое .loc, .iloc?





.loc, .iloc —

это атрибуты, которые используются для написания продуктивного кода при обработке большого объема данных.



Индексация в DataFrame

```
df['one'] # Если в качестве индекса указать имя столбца, получится одномерный набор данных типа Series.  
df.one   # К столбцу можно обращаться как к полю объекта, если имя столбца позволяет это сделать (= является строкой)  
df['one'].index # Индексы полученного одномерного набора данных.  
df['one'].name  # Можно получить имя данного столбца  
df['one']['d']  # Получение элемента массива. Сначала надо указать столбец, потом строку  
df['c']['one']  # А наоборот нельзя!
```

Правила индексации в pandas

```
df['b':'d'] # правая граница включена!  
df[1:3] # но диапазон целых чисел даёт диапазон строк с такими номерами, не включая правую границу (как  
обычно при индексировании списков).  
df[1] # при этом обратиться к индексу по номеру нельзя, а вот срез взять можно!
```

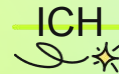


```
df[['one', 'three']]  
display(df['one'])  
display(df[['one']]) # как вы думаете, чем обусловлено отличие в представлении этой и прошлой строки?
```

Атрибут loc

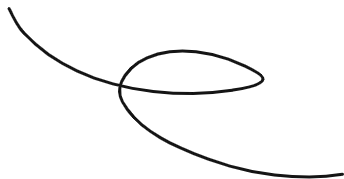
```
df.loc['b'] # __index__  
df.loc['b', 'one']  
df['one']['b'] # эта строка выведет то же что и предыдущая  
df.loc['a':'b', 'one'] # можно использовать срезы по-всякому  
df.loc['a':'b', :]  
df.loc[:, 'one'] # двоеточие, напомним, означает, что берем все значения по соответствующей размерности  
df.iloc[2] # атрибут iloc подобен loc: первый индекс — номер строки, второй — номер столбца. Это целые числа,  
конец диапазона не включается как обычно в питоне  
df.iloc[1:3]  
df.iloc[1:3, 0:2] # и тут срезы абсолютно валидны
```

Полезно знать



Булевская индексация —

это **выбор строк** с заданным условием.



Булевская индексация

```
df[df.three > 0]  
df[df.notna()]
```

Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый

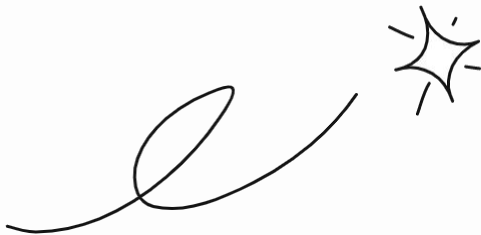


ЗАДАНИЕ ДЛЯ ЗАКРЕПЛЕНИЯ



Задание

Завершить работу над задачами с практикумов 3 и 4.





Задание

1. Задайте массив `grades` из оценок студентов (оценки от 0 до 100, количество элементов – по количеству студентов в вашей группе). Напишите программу, которая увеличит все оценки на 10%. ○ ●
2. Задайте массив `prices` из цен на товары (цены от 1 до 100 евро, количество – не менее 10). Сгенерируйте массив из булевых значений: `False` – для цен меньше 40 евро, `True` – для всех остальных. С его помощью сделайте скидку 5% для товаров с ценой не ниже 40.



Задание

3. Пусть у вас есть данные о ежедневных изменениях стоимости акций компании в течение недели. Найдите изменение цены за каждый день, а также дни с минимальным и максимальным изменением.

(Подсказка: Не забудьте, что когда вы ищите изменение в ценах, то кол-во значений уменьшается на 1, т.к. первое значение не с чем сравнивать.)

Пример вывода:

Изменения цен за день: [5 5 -8 -4 17 5]

Лучший день: суббота

Худший день: четверг





Задание

4. Пять спортсменов участвовали в соревновании, в котором нужно было бежать 2 часа, плавать 1 час и ехать на велосипеде 5 часов.
- Первый спортсмен бежал со скоростью 15 км/ч, плыл со скоростью 4 км/ч и ехал на велосипеде со скоростью 25 км/ч.
- Соответствующие скорости второго спортсмена: 14 км/ч, 5 км/ч, 27 км/ч.
- Третий спортсмен: 16 км/ч, 5 км/ч, 23 км/ч.
- Четвертый: 13 км/ч, 5.5 км/ч, 26 км/ч.
- Пятый: 12 км/ч, 4.5 км/ч, 30 км/ч.
- Побеждает тот, кто преодолел большее расстояние. Напишите программу, которая определяет победителя.
- (Подсказка: Подумайте, что здесь можно записать матрицей, что вектором, какая матричная операция определяет пройденное расстояние.)



Задание

5. Сделайте срез матрицы из предыдущей задачи, чтобы получить квадратную матрицу (т.е. оставляем только трех спортсменов). Сгенерируйте случайный столбец трех чисел от 100 до 200. Будем считать, что это расстояния, пройденные каждым спортсменом. Найдите определитель полученной матрицы и матрицу, обратную к ней. Умножьте обратную матрицу на случайный столбец (с помощью матричного умножения). Поясните, что вы нашли с помощью такой операции.
6. Транспонируйте матрицу из задачи 4. Поясните смысл каждой строки новой матрицы. На вектор какого размера ее можно умножить (дайте ответ для вектора-столбца и вектора-строки, для поэлементного умножения и для матричного умножения). Объясните практический смысл каждой операции.



Задание

1. Напишите программу, которая генерирует Series с данными о ежедневных доходах от трейдинга за прошлый год (индексы – это даты, суммы дохода – случайные, но учтите, что они могут быть и отрицательными).

Программа должны вывести:

- Годовую доходность.
- Среднюю дневную доходность.
- Медианную дневную доходность.
- Наивысшую дневную доходность.
- Наибольший дневной убыток.



Задание

2. Для Series из предыдущей задачи создайте срез с ежедневными доходами в мае. Выведите для мая:

- Месячную доходность.
- Среднюю дневную доходность.
- Медианную дневную доходность.
- Наивысшую дневную доходность.
- Наибольший дневной убыток.



Задание

3. Напишите программу для генерации DataFrame с информацией о 35 сотрудниках компании. Фамилии, должности и отделы должны выбираться случайным образом из заранее заданного списка. Зарплата генерируется как случайное число с нормальным распределением. Необходимо определить минимальную, максимальную и медианную зарплату. Вывести данные о сотрудниках с найденными зарплатами.
4. Для DataFrame из предыдущей задачи отсортируйте данные по размеру зарплаты. Из отсортированного датафрейма выведите 5 первых строк и 3 последних.





Задание

5. Напишите программу, которая:
 - Генерирует случайные данные о ежедневных объемах продаж ноутбуков, планшетов и смартфонов в магазинах A, B, C, D, E, ..., X, Y, Z.
 - Выведите описательные статистики по товарам и по магазинам.
6. Для датафрейма из предыдущей задачи запросите у пользователя ввод товара. Выведите все данные о продажах этого товара. Затем запросите у пользователя магазин, выведите данные о продажах товаров в этом магазине.
7. В предыдущей задаче найдите средний объем продаж выбранного товара. Выведите датафрейм с магазинами, объем продаж этого товара в которых был выше среднего.

Стать IT-шником может каждый



РАЗБОР ДОМАШНЕГО ЗАДАНИЯ





Домашнее задание

1. Напишите программу, которая:

- Генерирует два больших одномерных массива с 1 миллионом элементов каждый, заполненных случайными числами.
- Сначала выполняет поэлементное сложение этих массивов, используя циклы Python.
- Затем выполняет ту же операцию, используя возможности NumPy.
- Измеряет и выводит время выполнения каждого подхода.

Пример вывода:

Время выполнения сложения с использованием цикла Python: 0.5 секунды.

Время выполнения сложения с использованием NumPy: 0.01 секунды.



Домашнее задание

2. Создать большой двумерный массив и исследовать, как изменение порядка (C-style и Fortran-style) влияет на производительность операций сложения и умножения. Можно использовать тот же бенчмарк что и в лекции. Необходимо погуглить про то, как завести массив с порядком в Fortran-style. Не забывайте про magic в jupyter.



Домашнее задание

3. Напишите программу, которая:

- Запрашивает у пользователя размер квадратной матрицы N .
- Генерирует квадратную матрицу размером $N \times N$, заполненную случайными числами от -10 до 10 .
- Выводит сгенерированную матрицу на экран.
- Вычисляет и выводит определитель матрицы.
- Если определитель не равен нулю, программа должна вычислить и вывести обратную матрицу. В противном случае, программа должна сообщить, что обратной матрицы не существует.



Домашнее задание

1. Напишите программу, которая:
 - Запрашивает у пользователя ввод чисел, разделенных пробелами.
 - Преобразует введенные данные в объект Series, используя в качестве индексов буквы алфавита начиная с 'a'.
 - Выводит созданный объект Series на экран.



Домашнее задание

2. Напишите программу, которая:

- Запрашивает у пользователя данные для создания DataFrame. Данные включают в себя имена столбцов и строки значений, вводимые через пробел.
- Создает DataFrame с использованием введенных данных.
- Выводит созданный DataFrame на экран.
- Запрашивает у пользователя имя столбца, по которому необходимо отфильтровать данные.
- Выводит значения указанного столбца.



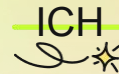


Домашнее задание

3. Напишите программу, которая:

- Запрашивает у пользователя количество строк данных для ввода.
- Для каждой строки запрашивает данные по трем столбцам: "Имя", "Возраст", "Регион", разделённые пробелом.
- Создает DataFrame на основе введенных данных.
- Выводит общую информацию о DataFrame, включая количество записей, средний возраст, и количество уникальных регионов.
- Выбирает и выводит на экран данные о самом молодом участнике.

Стать IT-шником может каждый



ОСТАВШИЕСЯ ВОПРОСЫ



Стать IT-шником может каждый



ЗАКЛЮЧЕНИЕ

